

Especificação do mini-projeto de Algoritmos e Programação

O mini-projeto da disciplina de Algoritmos e Programação do curso de Engenharia de Computação tem como objetivo proporcionar ao aluno a aplicação dos conceitos vistos na disciplina em temática encontrada no cotidiano das pessoas.

O mini-projeto deve ser desenvolvido em dupla e utilizar como linguagem de programação a linguagem Python.

É de suma importância que todos os itens constantes na especificação do miniprojeto sejam contemplados e que o desenvolvimento não faça uso de ferramentas de Inteligência Artificial (IA). Contamos com o compromisso e a responsabilidade de cada um!

- Data de entrega do miniprojeto: xx/xx/xxxx
- Local: AVA Moodle
- O que entregar:
 - Relatório resumido do desenvolvimento do projeto informando, inclusive, o que cada aluno desenvolveu no projeto.
 - Incluir no relatório o link para a pasta do miniprojeto (pasta que contém todos os arquivos) e que deve estar compartilhada com o professor.
 - Incluir no relatório o link para o youtube (usar modo “não listado”: link do vídeo “pitch” da apresentação do mini-projeto, com duração entre 3 e 5 minutos. Todos os integrantes devem estar no vídeo.

Tema: Campo Minado

Descrição

O objetivo do programa é desenvolver uma versão simplificada do jogo Campo Minado utilizando os conceitos vistos na disciplina. O usuário deve ser capaz de escolher um apelido, a dimensão do campo quadrado e a quantidade de bombas presentes no campo. O campo deverá ser gerado automaticamente, distribuindo as bombas em posições aleatórias da matriz. As demais posições deverão armazenar a quantidade de bombas existentes nas casas adjacentes.

Durante a execução:

- O jogador deverá visualizar o campo parcialmente oculto;
- O programa deverá exibir linhas e colunas de guia;
- O jogador deverá informar a posição que deseja revelar.

Após cada jogada, o campo atualizado deverá ser exibido novamente.

Regras do jogo

- Caso o jogador selecione uma posição contendo uma bomba, o jogo deverá ser encerrado com derrota.
- Caso todas as posições sem bombas sejam reveladas, o jogador vence a partida.
- Caso uma posição contendo 0 bombas adjacentes seja aberta, o programa deverá expandir automaticamente as posições vizinhas correspondentes.

Funcionalidades

As seguintes funcionalidades devem ser implementadas utilizando funções:

- Geração do campo;
 - O campo pode ter dimensões de no mínimo 2x2 e no máximo 9x9 – A entrada pode ficar em loop até que seja digitado um valor válido.
- Cálculo dos números adjacentes;



			1	1
1	1	1	2	3
3	3	3	2	2
2	3	3	2	1

Cada posição do campo que **não contém uma bomba** deve armazenar a quantidade de bombas existentes ao seu redor.

O número 3 indica que há 3 bombas nos 8 espaços adjacentes a ele, assim como o 1 indica apenas uma bomba.

Observe que há espaços vazios (ou com 0) que indicam não haver bombas ao seu redor – Raramente é possível encontrar um 8.

- Exibição do campo;
 - Acima de cada coluna deve haver um número correspondente ao número da coluna
 - Ao lado de cada linha deve haver um número correspondente ao número da linha
 - **Trapaça:** Caso o **Nome** do jogador seja “**Dev**”, também deverá ser exibido o campo com as bombas reveladas ao lado do campo original – Uma trapaça de jogo.

- Abertura de células;
 - A entrada deve ser dada em uma linha

Exemplo:

```
>>> (Linha Coluna): 3 5
```

- Expansão da cadeia de zeros.
 - Quando o jogador abre uma posição contendo **0**, o jogo deve revelar automaticamente as posições vizinhas.
 - O processo deve continuar enquanto existirem novas posições **0** sendo abertas. Quando não houver mais **0** 's conectados para expandir, o processo encerra.

Dica: Uma posição não deve ser aberta mais de uma vez, e sempre verifique se os índices permanecem dentro dos limites da matriz.

Requisitos

- O campo deve utilizar matrizes ([listas bidimensionais](#)).
- O programa deve utilizar estruturas de repetição e condicionais.
- Não é permitido utilizar bibliotecas externas além do módulo [random](#).
 - Buscar por “[from random import sample](#)” pode ajudar na geração aleatória de bombas por linha.
- O código deve possuir comentários e nomes de variáveis legíveis.
- O jogo deve permanecer em execução até vitória ou derrota.

Observações

- A interface do jogo será textual (**terminal**).
- Não é necessário implementar bandeiras, cronômetro ou níveis de dificuldade.
- A expansão da cadeia de zeros pode ser feita com **repetição ou recursividade**.

Exemplos de uso

Entrada:

```
Campo Minado
Seu nome: Gui
Medida do campo: 7
Nº de bombas (7-49): 20
```

- A primeira linha de entrada recebe o Nome do jogador (**str**);
- A segunda recebe uma medida **M** de 2 à 9 para o comprimento do quadrado (**int**);
- A terceira recebe o número (**int**) de bombas (de **M** à **M * M**) distribuídas no campo.

Saída:

Exemplo 1:

#	1	2	3	4	5	6	7
1	■	■	■	■	■	■	■
2	■	■	■	■	■	■	■
3	■	■	■	■	■	■	■
4	■	■	■	■	■	■	■
5	■	■	■	■	■	■	■
6	■	■	■	■	■	■	■
7	■	■	■	■	■	■	■
L	C:	7	1				
#	1	2	3	4	5	6	7
1	■	■	■	■	■	■	■
2	■	■	■	■	■	■	■
3	■	■	■	■	■	■	■
4	■	2	1	2	■	■	■
5	■	1	0	1	2	■	■
6	1	1	0	0	2	■	■
7	0	0	0	0	2	■	■
L	C:						

- A saída consiste nas linhas de grade, na matriz e na entrada do usuário;
- Como a matriz definida foi de tamanho 7, as linhas lateral e superior vão de 1 → 7;
- Observe que foi selecionado a Linha 7 e Coluna 1 – **que correspondia a um 0** – e foi exibido o mapa com algumas células reveladas – e segue até o jogador perder ou vencer.

Exemplo 2:

```
Campo Minado
Seu nome: Dev
Medida do campo: 5
Nº de bombas (5-25): 5
# 1 2 3 4 5
1 ■ ■ ■ ■ ■ 0 1 • 2 1
2 ■ ■ ■ ■ ■ 1 2 2 2 •
3 ■ ■ ■ ■ ■ 2 • 2 1 1
4 ■ ■ ■ ■ ■ 2 • 2 1 1
5 ■ ■ ■ ■ ■ 1 1 1 1 •
L C: |
```

- Aqui, observe que o nome do jogador é **Dev**, então a **trapaça** foi ativada.
- Ao lado do campo, foi exibido também o campo com os números e bombas já revelados. Assim, é possível ver que existem bombas nas posições (3 2), (4 2), (1 3), (2 5) e (5 5) – Isso facilita verificar se as bombas e os números estão sendo gerados corretamente.